Identifying a Data Gap

How AgenticSys decides that missing data is a *finding* rather than a mistake or a failure — and why the three get opposite responses. Reflects the codebase as of commit `741c617`.

1. Three things that all look like “no data”

Almost every failure in this area comes from collapsing these into one. They are not degrees of the same thing — each licenses a different sentence in the final answer, and confusing them produces claims that are stronger than the truth.

State	What happened	Correct response
ABSENT	The specialist never ran, timed out, or blew its turn budget. No query was issued.	Say **nothing** about that domain. Not even a negative — a negative claim is still a claim.
UNTESTED	A query ran but could not have answered the question: wrong column, wrong table, a filter condition that matches nothing.	Fix and re-query. **Not** a data gap — reporting it as one abandons data the case actually has.
EMPTY	The tool worked. The column exists, the rows are there, every value is blank. This case genuinely has no such data.	**This** is the data gap. Record it and move on; re-issuing returns the same thing.

A fourth case sits alongside them and is easy to miss: a query that ran, matched rows on every condition, and still returned nothing. That is a **real negative** — a finding worth reporting *with its search space*, not a gap.

2. Layer 1 — the tool decides, deterministically

The design principle is explicit in the source: rules the skill would otherwise have to remember are better as feedback the tool simply gives. Each signal below is emitted by the data tools with no LLM involved.

Signal in the tool result	State	Instruction carried in the message
`DATA GAP: column 'X' is EMPTY for this case`	**EMPTY**	“The tool worked… Record it as a data_gap and move on.”
`COLUMN NOT FOUND: 'X' is not a column of 'T'`	**UNTESTED**	“Call search_columns… **Do NOT report this as a data gap.**”
`column_is_empty: true`	**EMPTY**	Same state as row 1, seen through a *filter* column rather than a value column.
`_conditions_alone → testable: false`	**UNTESTED**	A condition names a column this table lacks. “NOT A NEGATIVE FINDING… use transaction_detail or join_table.”
`_conditions_alone → one condition matches 0 alone`	**UNTESTED**	That condition is the culprit. “Do NOT report this as ‘no such rows exist’.”
`_conditions_alone → all match alone`	**REAL NEGATIVE**	The emptiness is the combination. Report it with the counts as the search space.
`not_tested: true (sequence_join)`	**UNTESTED**	One side’s filter matched 0 rows, so no pair could exist regardless of the data.
`[FAILED <name>] <error_type>: …`	**ABSENT**	The specialist produced no output at all. The orchestrator may say nothing about that domain.

Why the first two must never be conflated: the DATA GAP wording says *do not retry*. Apply it to a simple name miss and the specialist abandons a variable the case really has. Apply the “check the column name” wording to a genuinely empty column and it burns rounds chasing a name that was already correct.

3. Layer 2 — the specialist's judgment

Beyond the deterministic signals, `data_query.md` instructs a specialist to declare a gap when:

- **Uncertainty** — "uncertainty → data_gaps, never plausible filler".
- **A tool errored and could not be recovered** — retry or declare a gap; never answer around it.
- **Coverage is narrower than asked** — the actual date range is shorter than the requested window.
- **Another domain owns the column** — e.g. a balance question reaching spend_payments defers to crossbu rather than substituting a spend figure.
- **Rounds ran out** — emit a partial answer with the unreached part named, rather than nothing.
- **An investigation direction could not be made concrete** — that is a gap, not a finding.
- **Precision limits** — sequence_join is day-grain, so an intraday question names that limit.
- **Null share ≥ 5%** in a group breakdown — disclose the excluded subset.

One filter governs all of it: **flag only gaps that materially affect THIS answer**. The schema enforces the brevity that implies — `SpecialistOutput.data_gaps` is capped at one bullet of ≤15 words, explicitly "do NOT enumerate generic data-quality concerns".

4. Layer 3 — the reviewer classifies it

The cross-specialist reviewer promotes the specialist's free-text note into a typed record:

```
class DataGap(BaseModel):
    specialist: str            # who hit it
    missing_data: str          # what was missing
    absence_interpretation: str
    is_signal: bool            # is the ABSENCE itself evidence?
```

is_signal is the load-bearing field: *is the absence itself evidence?* No collections call notes because the case was never escalated is signal. A column the vendor does not populate is not. Only signal-bearing gaps surface as flags on the final answer, and several of them together are one of the triggers for a `data_pull_request`.

5. The procedure, end to end

```
Did the specialist return output at all?
  NO  -> ABSENT     say nothing about that domain; flag the failure
  YES -> did a tool NAME a state?
         COLUMN NOT FOUND | testable:false | a zeroing condition
                        -> UNTESTED       fix and re-query; NOT a gap
         DATA GAP | column_is_empty
                        -> EMPTY          declare the gap; do NOT retry
         nothing named, 0 rows, every condition matched alone
                        -> REAL NEGATIVE  report it WITH the search space
         data returned
                        -> a FINDING      anything material unreachable?
                                          if so: ONE data_gaps line (<=15w)
```

A cross-check sits on top of this in synthesis: if a live-data null **contradicts** a specific claim in the curated reports, it is not publishable as a negative. The report has handed the team a concrete, checkable test case — run it before answering.

6. Where this is still weak

Layers 1 and 3 are deterministic and typed. **Layer 2 is the soft one**: "materially affects THIS answer" is pure judgment, compressed into fifteen words, and nothing verifies that a gap which *should* have been declared was. The observed failure mode is a specialist that had no usable column, declared no gap, and reported a confident negative instead. The tools now catch several specific shapes of that, but the general question — *should I have flagged a gap here?* — has no check on it.

A second, smaller risk: three separate places now implement the ABSENT/EMPTY distinction with hand-written strings (summarize_trend, aggregate_column, _zero_match_diagnostic). They agree today. Nothing keeps them in step.

Sources: `tools/data_tools.py` (_zero_match_diagnostic, _aggregate_column_impl, _summarize_trend_impl, _sequence_join_impl) · `skills/workflow/data_query.md` · `skills/workflow/synthesis.md` · `models/types.py` (SpecialistOutput, DataGap) · `agent_factories/agent_tools/agent_tool.py` (_record_failure).